

# DM580: Functional Programming

Welcome and Course Introduction

Casper Bach

# What is Functional Programming?

# What is Functional Programming?

- An established discipline for programming, using functions

# What is Functional Programming?

- An established discipline for programming, using functions
- Functions are a means to an end ...

# What is Functional Programming?

- An established discipline for programming, using functions
- Functions are a means to an end ...
- ... Namely, correct software that is easy to develop and maintain

# After Following This Lecture, You Will Be Able to:

- Explain what functional programming is
- Explain the motivation for functional programming
- Write your first functional program in Haskell

# Who are You?

# Who are You?

- CS BSc students; AI BSc students; minor subject students (“sidefagsstuderende”); others?

# Who are You?

- CS BSc students; AI BSc students; minor subject students (“sidefagsstuderende”); others?
- What programming languages do you know and have you used?
  - Python? Java? Scala? Lisp/Scheme? Haskell? Lean?

# Who are We?

# Who are We?

- Lecturer:
  - Casper Bach [[casperbach@imada.sdu.dk](mailto:casperbach@imada.sdu.dk)], associate professor of CS, FORM / IMADA, SDU
  - I teach FP and Advanced FP
  - My research are is *programming languages*. Topics include *functional programming*, *type and effect systems*, and *safe-by-construction code refactoring*.

# Who are We?

- Lecturer:
  - Casper Bach [[casperbach@imada.sdu.dk](mailto:casperbach@imada.sdu.dk)], associate professor of CS, FORM / IMADA, SDU
  - I teach FP and Advanced FP
  - My research are is *programming languages*. Topics include *functional programming*, *type and effect systems*, and *safe-by-construction code refactoring*.
- TAs:
  - Valdemar Bang Reib
  - William Senning

# After Following This Lecture, You Will Be Able to:

- Explain what functional programming is
- Explain the motivation for functional programming
- Write your first functional program in Haskell

# After Following This Lecture, You Will Be Able to:

- Explain what functional programming is
  - A paradigm for programming that uses functions for computation
- Explain the motivation for functional programming
- Write your first functional program in Haskell

# After Following This Lecture, You Will Be Able to:

- Explain what functional programming is
  - A paradigm for programming that uses functions for computation
- Explain the motivation for functional programming
  - Writing safe and correct software
- Write your first functional program in Haskell

# After Following This Lecture, You Will Be Able to:

- Explain what functional programming is
  - A paradigm for programming that uses functions for computation
- Explain the motivation for functional programming
  - Writing safe and correct software
- Write your first functional program in Haskell
  - Later in this lecture!

A large, dark gray, stylized lambda symbol ( $\lambda$ ) is positioned on the left side of the image, partially overlapping the text.

**What are the Benefits  
of Functional Programming?**

# Do More with Less

## Example Java Code

```
// Imperative.  
// Three passes, two intermediates.
```

```
List<Integer> temp1  
    = new ArrayList<>();  
for (Integer x : input)  
    if (x > 0) temp1.add(x);
```

```
List<Integer> temp2  
    = new ArrayList<>();  
for (Integer x : temp1)  
    temp2.add(x * x);
```

```
int sum = 0;  
for (Integer x : temp2)  
    sum += x;
```

```
// Functional.  
// Fused: single pass, no intermediates.
```

```
int sum = input.stream()  
    .filter(x → x > 0)  
    .map(x → x * x)  
    .reduce(0, Integer::sum);
```

# Do More with Less

## Example Java Code

```
> javac SimpleBenchmark.java
> java SimpleBenchmark
Warming up JVM...
```

Measuring...

≡ RESULTS ≡

Imperative: 0.49ms avg

Functional: 0.26ms avg

tFun/tImp: 0.53

# **Benefits of Functional Programming**

## **And One Downside**

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions
  - Benefit 1: types alone can faithfully convey the “contract” of a function

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions
  - Benefit 1: types alone can faithfully convey the “contract” of a function
  - Benefit 2: we can use standard mathematics (algebra+induction) to prove our software correct

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions
  - Benefit 1: types alone can faithfully convey the “contract” of a function
  - Benefit 2: we can use standard mathematics (algebra+induction) to prove our software correct
  - Benefit 3: enables aggressive compiler optimizations (e.g., fusion)

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions
  - Benefit 1: types alone can faithfully convey the “contract” of a function
  - Benefit 2: we can use standard mathematics (algebra+induction) to prove our software correct
  - Benefit 3: enables aggressive compiler optimizations (e.g., fusion)
  - Benefit 4: code (functions) seamlessly compose

# Benefits of Functional Programming

## And One Downside

- Good OO design *encourages* a style that minimizes side effects via abstractions
- FP *enforces* a style where we have *minimal side effects* via abstractions
  - Benefit 1: types alone can faithfully convey the “contract” of a function
  - Benefit 2: we can use standard mathematics (algebra+induction) to prove our software correct
  - Benefit 3: enables aggressive compiler optimizations (e.g., fusion)
  - Benefit 4: code (functions) seamlessly compose
  - Downside: in most computer architectures, function calls are more expensive than mutation. Clever compilers can eliminate or alleviate this downside; but not always.



Typed FP is a Vocabulary  
for Computation

# **From “How” to “What”**

**FP Encourages Declarative Thinking**

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”
  - `total = 0`
  - `for n in numbers:`
  - `if n % 2 == 0:`
  - `total += n`

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”
  - `total = 0`
  - `for n in numbers:`
  - `if n % 2 == 0:`
  - `total += n`
- Functional asks: “What is the result?”

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”
  - `total = 0`  
  `for n in numbers:`  
    `if n % 2 == 0:`  
      `total += n`
- Functional asks: “What is the result?”
  - `sum (filter isEven numbers)`

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”
  - `total = 0`  
  `for n in numbers:`  
    `if n % 2 == 0:`  
      `total += n`
- Functional asks: “What is the result?”
  - `sum (filter isEven numbers)`
  - The code *is* the problem statement

# From “How” to “What”

## FP Encourages Declarative Thinking

- Imperative asks: “What steps do I take?”
  - `total = 0`
  - `for n in numbers:`
  - `if n % 2 == 0:`
  - `total += n`
- Functional asks: “What is the result?”
  - `sum (filter isEven numbers)`
  - The code *is* the problem statement
- Bugs hide between intent and procedure. FP narrows or eliminates the gap.

# **Functional Design = Composable Software**

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces
- For example, an image processor:

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces
- For example, an image processor:

rotate ~~~> crop ~~~> grayscale ~~~> resize

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces
- For example, an image processor:

rotate  $\rightsquigarrow$  crop  $\rightsquigarrow$  grayscale  $\rightsquigarrow$  resize

- Each stage is a *pure* function of type `Image → Image`

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces
- For example, an image processor:

rotate  $\rightsquigarrow$  crop  $\rightsquigarrow$  grayscale  $\rightsquigarrow$  resize

- Each stage is a *pure* function of type Image  $\rightarrow$  Image
- Testable on its own, swappable, reorderable

# Functional Design = Composable Software

- FP lets you build complex systems from composable pieces
- For example, an image processor:

rotate  $\rightsquigarrow$  crop  $\rightsquigarrow$  grayscale  $\rightsquigarrow$  resize

- Each stage is a *pure* function of type Image  $\rightarrow$  Image
- Testable on its own, swappable, reorderable
- Complexity lives in the primitives, not the wiring

# Types Guide Design

# Types Guide Design

- Write types; then fill the code in the “obvious” way

# Types Guide Design

- Write types; then fill the code in the “obvious” way
- Example 1: what do `fst` and `snd` do? (Hint: `a` and `b` are forall-quantified type variables)
  - `fst :: (a, b) → a`
  - `snd :: (a, b) → b`

# Types Guide Design

- Write types; then fill the code in the “obvious” way
- Example 1: what do `fst` and `snd` do? (Hint: `a` and `b` are forall-quantified type variables)
  - `fst :: (a, b) → a`
  - `snd :: (a, b) → b`
- Example 2: what does `map` do? (Hint: `a` and `b` are forall-quantified type variables)
  - `map :: (a → b) → [a] → [b]`

# Types Guide Design

- Write types; then fill the code in the “obvious” way
- Example 1: what do `fst` and `snd` do? (Hint: `a` and `b` are forall-quantified type variables)
  - `fst :: (a, b) → a`
  - `snd :: (a, b) → b`
- Example 2: what does `map` do? (Hint: `a` and `b` are forall-quantified type variables)
  - `map :: (a → b) → [a] → [b]`
- Types are extremely powerful: most mathematical statements can be expressed as *dependent types*. Proofs are then functional, *dependently typed* programs!
  - Many proof assistants (Lean, Agda, Rocq, etc.) are “just” functional programming languages
  - FP lets us write software that is *correct by construction*

# Equational Reasoning for Our Code

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere
- FP lets us apply this reasoning to our code.

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere
- FP lets us apply this reasoning to our code.
- Lets us refactor with confidence.

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere
- FP lets us apply this reasoning to our code.
- Lets us refactor with confidence.
  - For example, we can replace  $f(3)$  with  $9$  if we know that  $f(3) = 9$

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere
- FP lets us apply this reasoning to our code.
- Lets us refactor with confidence.
  - For example, we can replace  $f(3)$  with  $9$  if we know that  $f(3) = 9$
  - For example, fusion (the law that the prior Java example uses to optimize!)

# Equational Reasoning for Our Code

- In algebra, if  $a, b$  are expressions, and  $a = b$ , we can swap  $a$  for  $b$  anywhere
- FP lets us apply this reasoning to our code.
- Lets us refactor with confidence.
  - For example, we can replace  $f(3)$  with  $9$  if we know that  $f(3) = 9$
  - For example, fusion (the law that the prior Java example uses to optimize!)

$$\text{map } g \circ \text{map } f = \text{map } (g \circ f)$$

# **Why It Matters**

**And Why Programming Languages and the Industry Increasingly Utilize FP**

# Why It Matters

## And Why Programming Languages and the Industry Increasingly Utilize FP

| Concept        | What it means                            | Why it helps                       |
|----------------|------------------------------------------|------------------------------------|
| Pure functions | Same input gives the same output, always | Easy testing, predictable behavior |

# Why It Matters

## And Why Programming Languages and the Industry Increasingly Utilize FP

| Concept        | What it means                               | Why it helps                            |
|----------------|---------------------------------------------|-----------------------------------------|
| Pure functions | Same input gives the same output, always    | Easy testing, predictable behavior      |
| Immutability   | Once created, data structures do not change | Safe concurrency, fewer nasty surprises |

# Why It Matters

## And Why Programming Languages and the Industry Increasingly Utilize FP

| Concept        | What it means                               | Why it helps                            |
|----------------|---------------------------------------------|-----------------------------------------|
| Pure functions | Same input gives the same output, always    | Easy testing, predictable behavior      |
| Immutability   | Once created, data structures do not change | Safe concurrency, fewer nasty surprises |
| Composition    | Small pieces, clear assembly                | Modular, reusable architecture          |

# Why It Matters

## And Why Programming Languages and the Industry Increasingly Utilize FP

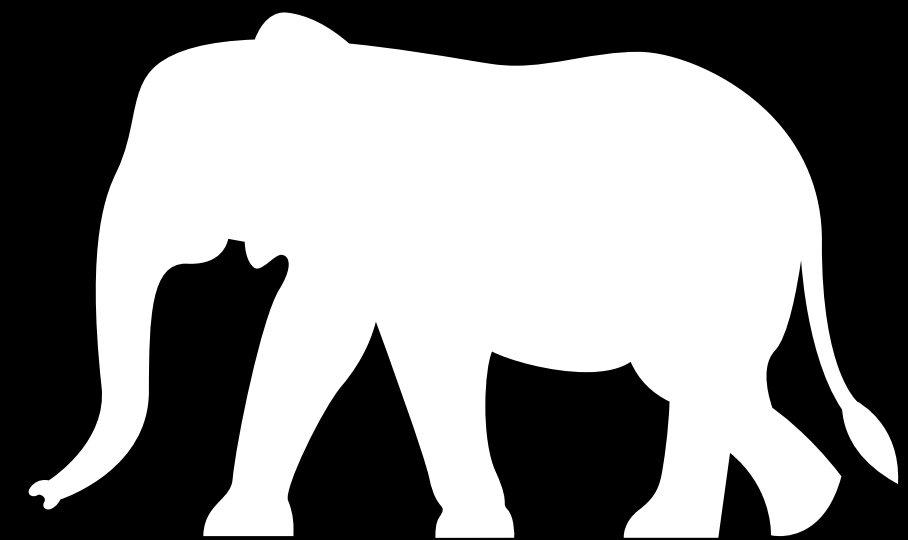
| Concept           | What it means                                  | Why it helps                                         |
|-------------------|------------------------------------------------|------------------------------------------------------|
| Pure functions    | Same input gives the same output, always       | Easy testing, predictable behavior                   |
| Immutability      | Once created, data structures do not change    | Safe concurrency, fewer nasty surprises              |
| Composition       | Small pieces, clear assembly                   | Modular, reusable architecture                       |
| Typed programming | Programs are subject to correctness invariants | Compilers automatically check and enforce invariants |

# Why It Matters

## And Why Programming Languages and the Industry Increasingly Utilize FP

| Concept              | What it means                                  | Why it helps                                         |
|----------------------|------------------------------------------------|------------------------------------------------------|
| Pure functions       | Same input gives the same output, always       | Easy testing, predictable behavior                   |
| Immutability         | Once created, data structures do not change    | Safe concurrency, fewer nasty surprises              |
| Composition          | Small pieces, clear assembly                   | Modular, reusable architecture                       |
| Typed programming    | Programs are subject to correctness invariants | Compilers automatically check and enforce invariants |
| Equational reasoning | Code = equations                               | Safe refactoring, restructuring, and optimization    |

Do We Need FP Now that  
LLMs Can Generate Code?



# **Functional Programming in the LLM Era**

**LLMs Can Generate Code for Us. Do We Still Need FP?**

# Functional Programming in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need FP?

- Functional code (LLM-generated or human-crafted) is less bug prone and *verifiable*

# Functional Programming in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need FP?

- Functional code (LLM-generated or human-crafted) is less bug prone and *verifiable*
  - We can *prove* (or even have an LLM prove) that our code is correct

# Functional Programming in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need FP?

- Functional code (LLM-generated or human-crafted) is less bug prone and *verifiable*
  - We can *prove* (or even have an LLM prove) that our code is correct
  - (Procedural and OO code is in principle also verifiable; but verification requires substantially more involved techniques and mathematics.)

# Functional Programming in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need FP?

- Functional code (LLM-generated or human-crafted) is less bug prone and *verifiable*
  - We can *prove* (or even have an LLM prove) that our code is correct
  - (Procedural and OO code is in principle also verifiable; but verification requires substantially more involved techniques and mathematics.)
- LLM-generated code is only as good as its instructions and the scrutiny applied to its output. FP literacy is a necessity.

# Functional Programming in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need FP?

- Functional code (LLM-generated or human-crafted) is less bug prone and *verifiable*
  - We can *prove* (or even have an LLM prove) that our code is correct
  - (Procedural and OO code is in principle also verifiable; but verification requires substantially more involved techniques and mathematics.)
- LLM-generated code is only as good as its instructions and the scrutiny applied to its output. FP literacy is a necessity.
- This course will make you FP literate

# **Why Learn FP in the LLM Era**

**LLMs Can Generate Code for Us. Do We Still Need to Learn FP?**

# Why Learn FP in the LLM Era

**LLMs Can Generate Code for Us. Do We Still Need to Learn FP?**

- Short answer: yes.

# Why Learn FP in the LLM Era

**LLMs Can Generate Code for Us. Do We Still Need to Learn FP?**

- Short answer: yes.
- The CS degree teaches *you* to think computationally

# Why Learn FP in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need to Learn FP?

- Short answer: yes.
- The CS degree teaches *you* to think computationally
- Functional programming provides an important framework for expressing and implementing computation

# Why Learn FP in the LLM Era

## LLMs Can Generate Code for Us. Do We Still Need to Learn FP?

- Short answer: yes.
- The CS degree teaches *you* to think computationally
- Functional programming provides an important framework for expressing and implementing computation
- *We* (computer scientists, programmers, humanity) must remain in control of how and what we compute. The future needs computational thinkers.

# Course LLM Policy

# Course LLM Policy

- LLMs can be great for augmenting your understanding of a topic. Asking questions about understanding is, in principle, fine. But we recommend:

# Course LLM Policy

- LLMs can be great for augmenting your understanding of a topic. Asking questions about understanding is, in principle, fine. But we recommend:
  - Consult the **book or a TA** before consulting an LLM.

# Course LLM Policy

- LLMs can be great for augmenting your understanding of a topic. Asking questions about understanding is, in principle, fine. But we recommend:
  - Consult the **book or a TA** before consulting an LLM.
  - Make sure you **validate any claims made by an LLM**. Hallucination and LLM pollution occur, even for well-established topics, such as FP.

# Course LLM Policy

- LLMs can be great for augmenting your understanding of a topic. Asking questions about understanding is, in principle, fine. But we recommend:
  - Consult the **book or a TA** before consulting an LLM.
  - Make sure you **validate any claims made by an LLM**. Hallucination and LLM pollution occur, even for well-established topics, such as FP.
  - The book for the course is excellent. **LLMs are by no means necessary to use for this course.**

# Course LLM Policy

- LLMs can be great for augmenting your understanding of a topic. Asking questions about understanding is, in principle, fine. But we recommend:
  - Consult the **book or a TA** before consulting an LLM.
  - Make sure you **validate any claims made by an LLM**. Hallucination and LLM pollution occur, even for well-established topics, such as FP.
  - The book for the course is excellent. **LLMs are by no means necessary to use for this course.**
- **Do not use LLMs to generate code in this course.** Generating code solutions to assignments inhibits learning. Passing off an LLM's work as your own constitutes fraud. A bit of pain is a natural part of learning.



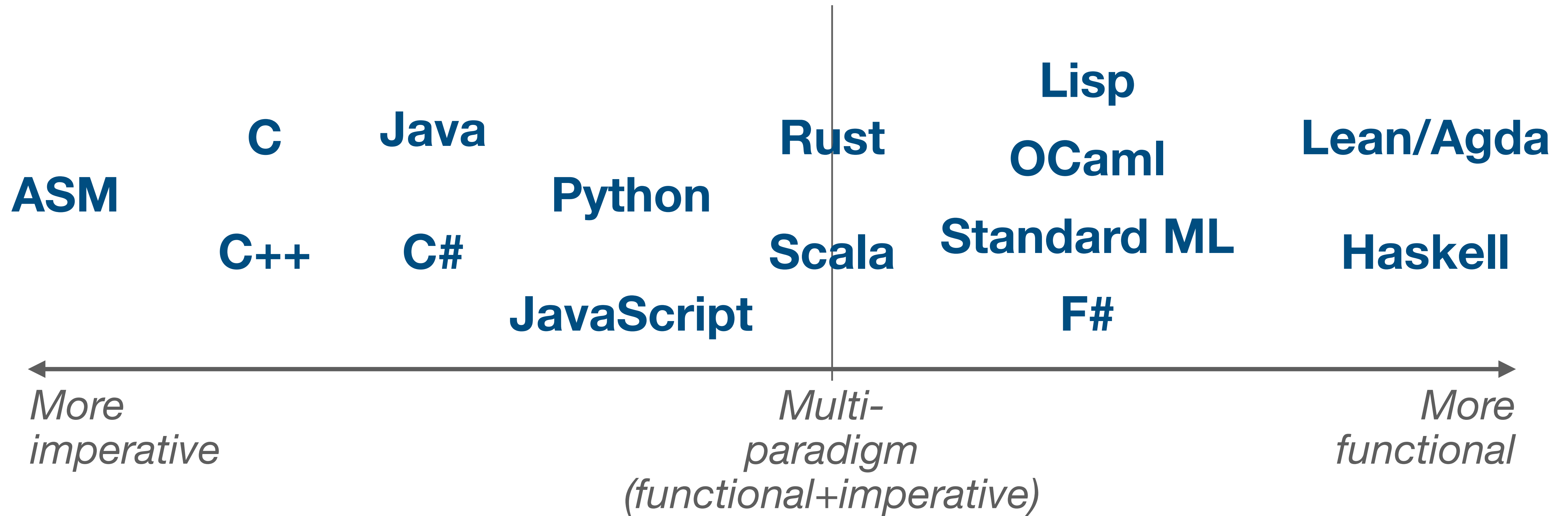
What is FP Used for In Practice?

# FP in the Industry

- Haskell for enterprise systems  
[at H-E-B] [at Mercury] [for FinTech] [for Blockchain]
- OCaml/OxCaml for FinTech  
[Jane Street] [OxCaml]
- Scala for enterprise systems  
[once upon a time at Twitter] [Disney+] [German critical infrastructure]
- More examples [Haskell wiki]

# FP is Supported by Virtually All Modern Languages

An Informal Plot Illustrating How “Imperative” vs. “Functional” Languages are\*



\* The idiomaticity of using FP varies between languages, communities, and applications.



# Programming in Haskell

# Why Use Haskell?

## Five Benefits

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., IO).

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., IO).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., IO).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`
  - ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., `IO`).

- ▶ `processTransactions :: [Transaction] → Either TError StateChange`
- ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`
- ▶ `runTransactions :: [Transaction] → IO ()`

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., `IO`).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`
  - ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`
  - ▶ `runTransactions :: [Transaction] → IO ()`
2. *Lazy evaluation* often strikes a balance between being *declarative and efficient*

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., `IO`).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`
  - ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`
  - ▶ `runTransactions :: [Transaction] → IO ()`
2. *Lazy evaluation* often strikes a balance between being *declarative and efficient*
3. *Type system* that rules out many common programming errors by construction

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., `IO`).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`
  - ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`
  - ▶ `runTransactions :: [Transaction] → IO ()`
2. *Lazy evaluation* often strikes a balance between being *declarative and efficient*
3. *Type system* that rules out many common programming errors by construction
4. Mature and sophisticated compiler that aggressively optimizes code

# Why Use Haskell?

## Five Benefits

1. *Pure computation* is the default. No side-effects, unless we use a special type (e.g., `IO`).
  - ▶ `processTransactions :: [Transaction] → Either TError StateChange`
  - ▶ `applyChange :: Ledger → StateChange → Either CError Ledger`
  - ▶ `runTransactions :: [Transaction] → IO ()`
2. *Lazy evaluation* often strikes a balance between being *declarative and efficient*
3. *Type system* that rules out many common programming errors by construction
4. Mature and sophisticated compiler that aggressively optimizes code
5. Syntax resembles many other FP languages; Haskell concepts such as *type classes* are now in many other languages (e.g., Rust)

# Why Use Haskell?

## Five Downsides

# Why Use Haskell?

## Five Downsides

1. *Steep learning curve.* Syntax differs from the C-like syntax that many other languages have adopted. Learning to think with *types* takes time and effort.

# Why Use Haskell?

## Five Downsides

1. *Steep learning curve.* Syntax differs from the C-like syntax that many other languages have adopted. Learning to think with *types* takes time and effort.
2. *Error messages* can be difficult to understand for beginners.

# Why Use Haskell?

## Five Downsides

1. *Steep learning curve.* Syntax differs from the C-like syntax that many other languages have adopted. Learning to think with *types* takes time and effort.
2. *Error messages* can be difficult to understand for beginners.
3. *Laziness can cause space leaks* that can be difficult to diagnose and fix.

# Why Use Haskell?

## Five Downsides

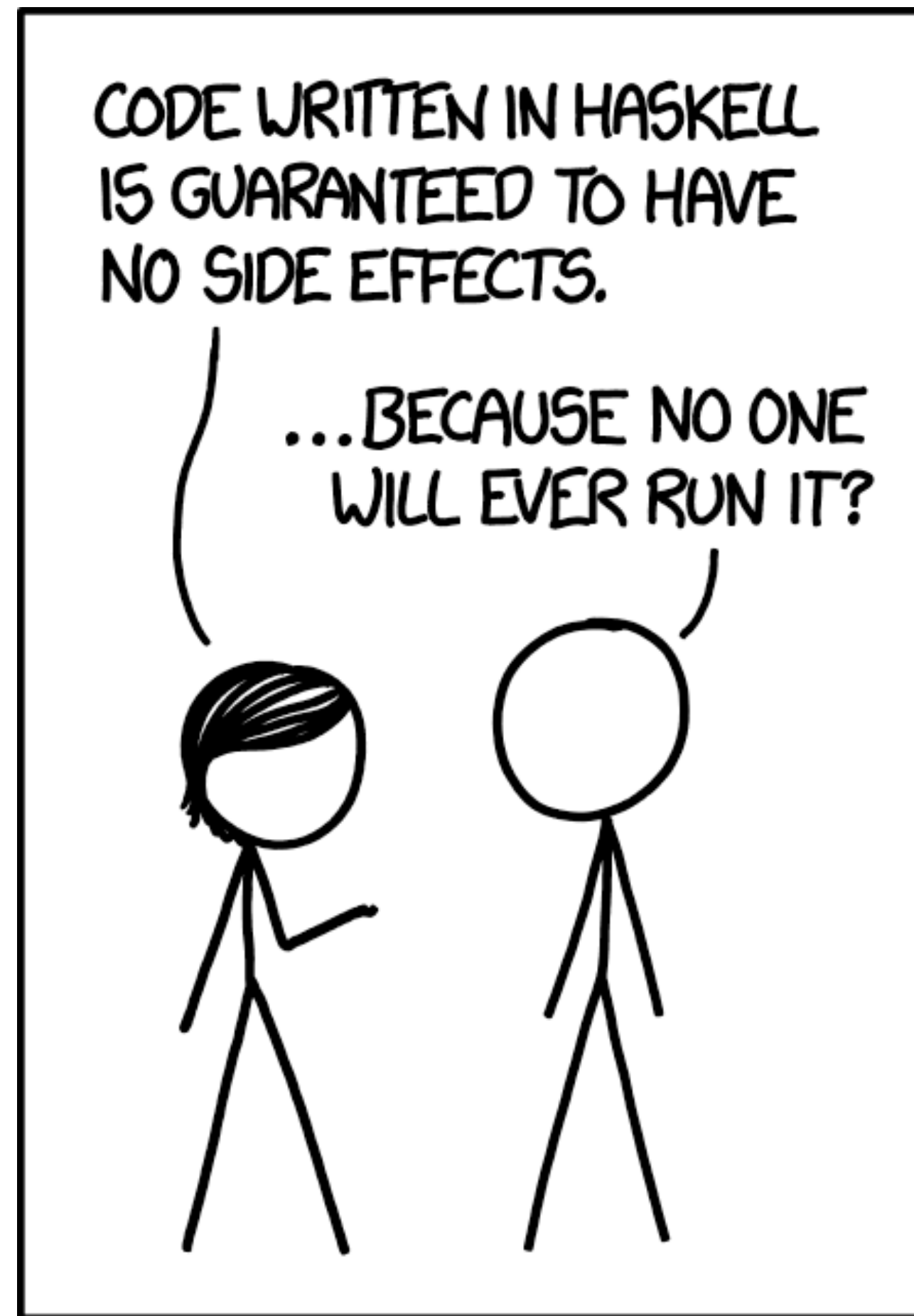
1. *Steep learning curve.* Syntax differs from the C-like syntax that many other languages have adopted. Learning to think with *types* takes time and effort.
2. *Error messages* can be difficult to understand for beginners.
3. *Laziness can cause space leaks* that can be difficult to diagnose and fix.
4. *Industrial adoption* of Haskell (not FP in general) is still somewhat limited.

# Why Use Haskell?

## Five Downsides

1. *Steep learning curve.* Syntax differs from the C-like syntax that many other languages have adopted. Learning to think with *types* takes time and effort.
2. *Error messages* can be difficult to understand for beginners.
3. *Laziness can cause space leaks* that can be difficult to diagnose and fix.
4. *Industrial adoption* of Haskell (not FP in general) is still somewhat limited.
5. *Purity as a strict default* makes some applications more difficult to write than they could be in more multi-paradigmatic languages.

# Less Used Than It Deserves



<https://xkcd.com/1312/>

# Haskell vs. Functional Programming

- Haskell is a vehicle for learning to think and write typed, functional code
- Typed functional programming as a concept transcends languages
- Advice: learn by doing. Follow team lessons; do the exercises.

A large, dark gray, stylized letter 'A' graphic that serves as a background element on the left side of the slide.

# Practical Information and Course Structure

# Course Structure

- One lecture per week, one team lesson per week
- The team lesson programming exercises will generally be about content covered the week before
- As part of the programming assignments, you will build a small project in increments. The project will prepare you for the final exam project.

# Course Plan and Reading

|                                                                                                                    |                                                                                              |                                                                                                |                                                                                            |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Week 39 (THIS WEEK)</b><br>Introduction and Haskell Preliminaries<br><i>Read:</i> * Foreword + 1 + 2            | <b>Week 40</b><br>Types and Basic Functions<br><i>Read:</i> 3 + 4                            | <b>Week 41</b><br>Lists and Recursion<br><i>Read:</i> 5 + 6<br><i>Submit mandatory assn. 1</i> | <b>Week 42</b><br><i>Fall Break</i>                                                        |
| <b>Week 43</b><br>Higher-Order Functions and Type Classes<br><i>Read:</i> 7 + 8<br><i>Submit mandatory assn. 2</i> | <b>Week 44</b><br>Programming with I/O<br><i>Read:</i> 10<br><i>Submit mandatory assn. 3</i> | <b>Week 45</b><br>Monads<br><i>Read:</i> 12                                                    | <b>Week 46</b><br><i>No lecture</i><br><i>Read:</i> TBA<br><i>Submit mandatory assn. 4</i> |
| <b>Week 47</b><br>Concurrency<br><i>Read:</i><br><a href="#">Beautiful Concurrency</a>                             | <b>Week 48</b><br>Lazy Evaluation<br><i>Read:</i> 15<br><i>Submit mandatory assn. 5</i>      | <b>Week 49</b><br>Exam Project<br><i>Read:</i> TBA                                             | <b>Week 50</b><br>Q&A Session                                                              |

\* Except where otherwise indicated, reading refers to Graham Hutton’s *Programming in Haskell*, second edition

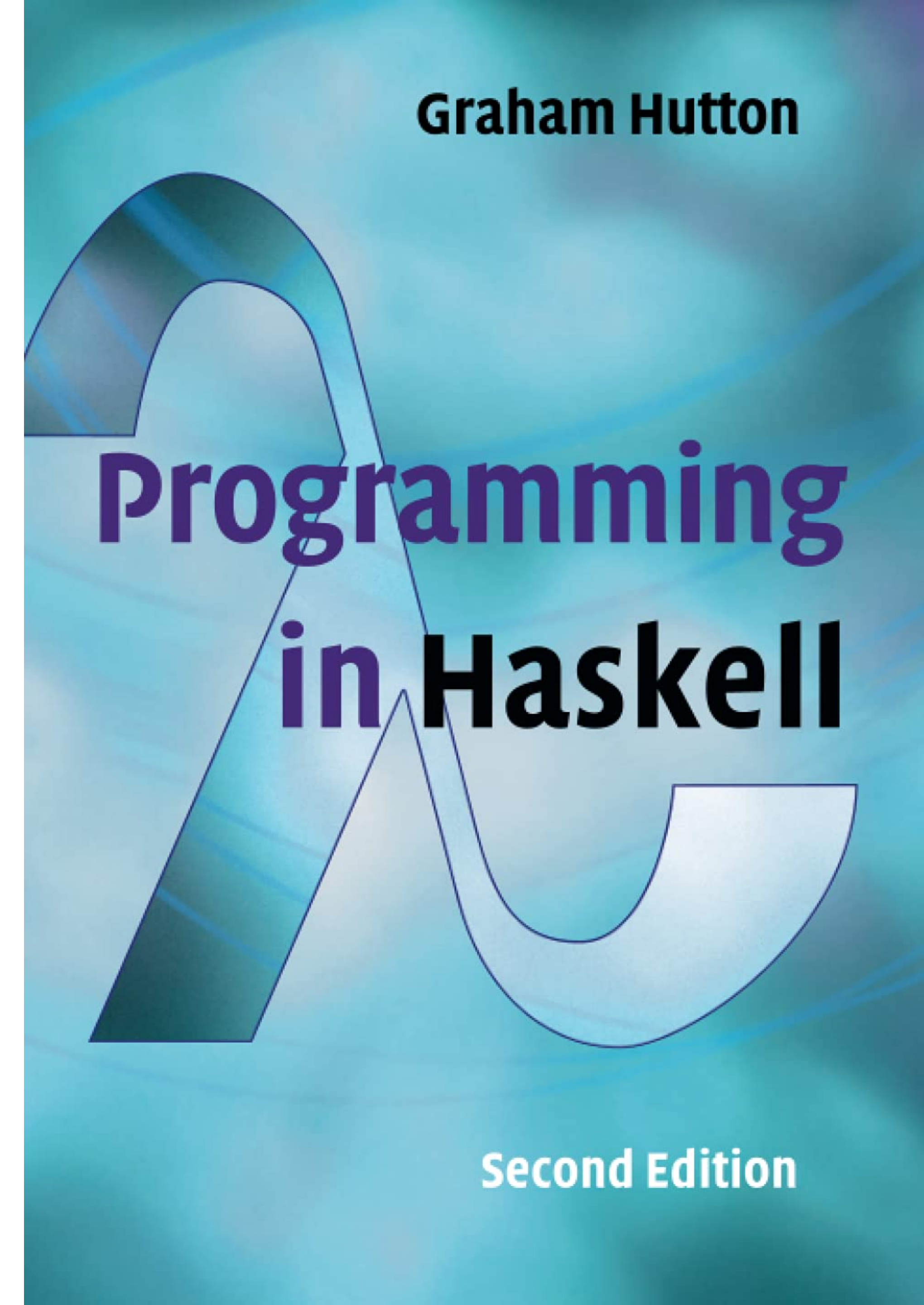
# Expected Workload

- 1 ECTS  $\cong$  25-30 hours; so 5 ECTS  $\cong$  125-150 hours
- 12 weeks until end of term; so 10.5-12.5 hours per week
- Rough weekly hours:
  - Lectures: ~2 hours per week
  - Team lessons: ~2 hours per week
  - Mandatory assignments: ~1 hour per week, for ~8 weeks
  - Self-study and reading: ~5.5-7.5

# Programming in Haskell

by Graham Hutton

- The main source of reading for this course
- Provides a gentle and pedagogical introduction to functional programming in Haskell
- Videos by Graham can be found here:  
<https://people.cs.nott.ac.uk/pszgmh/pih.html>



# Pass/Fail Assignments

- Five pass/fail assignments, submitted for feedback (no grade)
- Deadline: Fridays at 17:00. Submit via ItsLearning. Some weeks are submissionless; see which on ItsLearning.
- Purpose:
  - Feedback on your learning
  - Training for the exam project (also an *individual* project)
- Individual assignments. Fine to discuss with TAs and class mates, but complete and hand in individually. No auto-generated code.
- Expected weekly work load: ~1 hour. Some weeks have bigger assignments. Bigger assignments have longer deadlines.

# Team Lesson Exercises

- Team Lessons in each week
- Will generally revolve around the topic of the week before; e.g., exercises next week relate to Chapters 1+2 of the book
- Distinct from the pass/fail assignments

# Exam

- **Programming Project and Report**
  - Hand-in date: TBA — roughly one week before the oral exam
  - Project and report requirements: TBA — expected release: week 49\*
- **Oral exam** (15 minutes)
  - Expect questions about both project and curriculum in general
  - Focus is on assessing that you meet the course learning objectives

\* <https://ugenr.dk> lets you translate week numbers into dates

# Lectures

- Based on slides and code examples
- Reading before the lecture is recommended
- Will focus more on motivation and background, whereas team session exercises will teach you the techniques themselves. *Learning by doing.*



# Our First Program in Haskell

# After Following This Lecture, You Will Be Able to:

- ✓ Explain what functional programming is
  - A paradigm for programming that uses functions for computation
- ✓ Explain the motivation for functional programming
  - Writing safe and correct software
- Write your first functional program in Haskell

# From Python to Haskell

## Function Definitions

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

```
add :: Int → Int → Int  
add x y = x + y
```

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

```
add :: Int → Int → Int  
add x y = x + y
```

```
-- This is a one line Haskell comment.
```

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

```
add :: Int → Int → Int  
add x y = x + y
```

```
-- This is a one line Haskell comment.
```

```
{- And this is a multi-line comment. -}
```

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

```
add :: Int → Int → Int  
add x y = x + y
```

```
-- This is a one line Haskell comment.
```

```
{- And this is a multi-line comment. -}
```

```
{-  
Note: the type signature (the line with the  
`::`) is, in principle, optional. Unless you  
have good reasons to leave it out, it is good  
style to include type signatures.  
-}
```

# From Python to Haskell

## Function Definitions

```
def add(x, y):  
    return x + y
```

```
add :: Int → Int → Int  
add x y = x + y
```

```
-- This is a one line Haskell comment.
```

```
{- And this is a multi-line comment. -}
```

```
{-  
Note: the type signature (the line with the  
`::`) is, in principle, optional. Unless you  
have good reasons to leave it out, it is good  
style to include type signatures.  
-}
```

```
{-  
Discuss: is the type signature of `add`  
obvious? Why (not)?  
-}
```

# **Running Your First Haskell Program**

## **Using the REPL (Read-Evaluate-Print-Loop)**

# Running Your First Haskell Program

## Using the REPL (Read-Evaluate-Print-Loop)

```
$ ls -l
total 8
drwxr-xr-x  4 cbp  staff  128B Aug 13 13:57 .
drwxr-xr-x  4 cbp  staff  128B Aug 13 14:04 ..
-rw-r--r--  1 cbp  staff   16B Aug 13 13:57 PythonFunDefs.hs
```

# Running Your First Haskell Program

## Using the REPL (Read-Evaluate-Print-Loop)

```
$ ls -l
total 8
drwxr-xr-x  4 cbp  staff   128B Aug 13 13:57 .
drwxr-xr-x  4 cbp  staff   128B Aug 13 14:04 ..
-rw-r--r--  1 cbp  staff    16B Aug 13 13:57 PythonFunDefs.hs
$ ghci
GHCi, version 9.10.3: https://www.haskell.org/ghc/  :? for help
ghci>
```

# Running Your First Haskell Program

## Using the REPL (Read-Evaluate-Print-Loop)

```
$ ls -l
total 8
drwxr-xr-x  4 cbp  staff   128B Aug 13 13:57 .
drwxr-xr-x  4 cbp  staff   128B Aug 13 14:04 ..
-rw-r--r--  1 cbp  staff    16B Aug 13 13:57 PythonFunDefs.hs
$ ghci
GHCi, version 9.10.3: https://www.haskell.org/ghc/  :? for help
ghci>
ghci> :load PythonFunDefs.hs
[1 of 2] Compiling Main                ( PythonFunDefs.hs, interpreted )
Ok, one module loaded.
ghci>
```

# Running Your First Haskell Program

## Using the REPL (Read-Evaluate-Print-Loop)

```
$ ls -l
total 8
drwxr-xr-x  4 cbp  staff   128B Aug 13 13:57 .
drwxr-xr-x  4 cbp  staff   128B Aug 13 14:04 ..
-rw-r--r--  1 cbp  staff    16B Aug 13 13:57 PythonFunDefs.hs
$ ghci
GHCi, version 9.10.3: https://www.haskell.org/ghc/  :? for help
ghci>
ghci> :load PythonFunDefs.hs
[1 of 2] Compiling Main                ( PythonFunDefs.hs, interpreted )
Ok, one module loaded.
ghci>
ghci> add 1 2
3
ghci>
```

# From Python to Haskell

## Conditionals

# From Python to Haskell

## Conditionals

```
def describe(n):  
    if n > 0:  
        return "positive"  
    elif n < 0:  
        return "negative"  
    else:  
        return "zero"
```

# From Python to Haskell

## Conditionals

```
def describe(n):  
    if n > 0:  
        return "positive"  
    elif n < 0:  
        return "negative"  
    else:  
        return "zero"
```

```
describe :: Int → String  
describe n =  
    if n > 0  
    then "positive"  
    else if n < 0  
    then "negative"  
    else "zero"
```

# From Python to Haskell

## Loops and Recursion

# From Python to Haskell

## Loops and Recursion

```
def double_all(xs):  
    result = []  
    for x in xs:  
        result.append(x * 2)  
    return result
```

# From Python to Haskell

## Loops and Recursion

```
def double_all(xs):  
    result = []  
    for x in xs:  
        result.append(x * 2)  
    return result
```

```
-- Haskell, using a list comprehension  
doubleAll :: [Int] → [Int]  
doubleAll xs = [x * 2 | x ← xs]
```

# From Python to Haskell

## Loops and Recursion

```
def double_all(xs):  
    result = []  
    for x in xs:  
        result.append(x * 2)  
    return result
```

```
-- Haskell, using a list comprehension  
doubleAll :: [Int] → [Int]  
doubleAll xs = [x * 2 | x ← xs]
```

```
-- Equivalently, with `map`. (Arguably  
-- most idiomatic in Haskell.)  
doubleAll' :: [Int] → [Int]  
doubleAll' xs = map (* 2) xs
```

# From Python to Haskell

## Loops and Recursion

```
def double_all(xs):  
    result = []  
    for x in xs:  
        result.append(x * 2)  
    return result
```

```
-- Haskell, using a list comprehension  
doubleAll :: [Int] → [Int]  
doubleAll xs = [x * 2 | x ← xs]
```

```
-- Equivalently, with `map`. (Arguably  
-- most idiomatic in Haskell.)  
doubleAll' :: [Int] → [Int]  
doubleAll' xs = map (* 2) xs
```

```
-- Equivalently, using pattern matching  
-- and recursion. (More on this later in  
-- the course.)  
doubleAll'' :: [Int] → [Int]  
doubleAll'' [] = []  
doubleAll'' (x:xs) = x*2 : doubleAll'' xs
```

# From Python to Haskell

## Putting Things Together

```
def sum_positive_squares(numbers):  
    total = 0  
    for n in numbers:  
        if n > 0:  
            total = total + n * n  
    return total
```

```
sumPositiveSquares :: [Int] → Int  
sumPositiveSquares numbers =  
    sum [ n * n | n ← numbers, n > 0 ]
```

A large, dark gray, stylized lambda symbol ( $\lambda$ ) is positioned on the left side of the image, extending from the top to the bottom. It has a thick, rounded stroke.

Questions?